1. Bài toán và phạm vi thực hiện

1.1. Phần thực hiện của thành viên số 5

Thành viên số 5 — Võ Nhật Minh — MSSV: 24120202.

Bloxorz là bài toán tìm chuỗi hành động đưa khối tới trạng thái thắng. Hàm `core.game_rules.is_goal` xác định trạng thái đích: khối chưa tách và đứng đúng ô đích. Thành viên số 5 phụ trách cài đặt A*, xây dựng hàm ước lượng (heuristic) và đánh giá lời giải trên đồ thị chuyển trạng thái do phần lõi trò chơi cung cấp.

Trong phiên bản này, A* tối thiểu hóa tổng chi phí, không nhất thiết tối thiểu số hành động. Chi phí được tính bằng `StepCost` trong `solvers/common.py`. Hành động `SPACE` cũng được tính vào chuỗi lời giải và có chi phí.

1.2. Đầu vào và đầu ra

Đầu vào gồm màn chơi `Level` và trạng thái bắt đầu `GameState`, có thể là trạng thái ban đầu hoặc trạng thái hiện tại của người chơi. `GameState` bất biến và có thể dùng làm khóa của bảng băm. Nó chứa thông tin về khối và mặt nạ bit biểu diễn trạng thái các cầu. Thông tin khối gồm tư thế, trạng thái tách, vị trí hai khối lập phương và khối đang được điều khiển. Thuật toán dùng toàn bộ trạng thái làm khóa tìm kiếm.

Đầu ra là `SolverResult`, gồm chuỗi lời giải, trạng thái thành công, tổng chi phí, số hành động, số lần mở rộng và các số đo thời gian, bộ nhớ. Chuỗi lời giải dùng các ký hiệu `U`, `D`, `L`, `R`, `SPACE`. Nếu trạng thái ban đầu đã thắng, chuỗi rỗng là lời giải hợp lệ có chi phí bằng 0. Nếu hàng đợi tìm kiếm cạn, trường `path` là `None`, `solution_length` bằng -1 và `cost` bằng vô cực.

1.3. Ranh giới giữa các thành phần

Bảng 1: Phân chia trách nhiệm trong chương trình hiện tại.

Thành phần	Trách nhiệm
`core/interfaces.py`	Các kiểu trạng thái, màn chơi và kết quả dùng chung.
`core/game_rules.py`	Sinh trạng thái kế tiếp hợp lệ và kiểm tra đích.
`solvers/common.py`	Quy tắc tính chi phí và hàm đo thời gian, bộ nhớ.
`solvers/astar.py`	Quản lý tìm kiếm, hàm ước lượng và dựng lời giải.
`ui/solver_bridge.py`	Chạy thuật toán trong tiến trình riêng và trả kết quả về giao diện.

Phạm vi bảo đảm tính đúng

A* không tự cài lại luật công tắc, cầu hay tách và ghép khối. Tính tối ưu được xét trên đồ thị do phần lõi hiện tại cung cấp, với màn chơi giữ cố định trong mỗi lần giải. Việc phát lại một lời giải hợp lệ qua phần lõi không thay thế kiểm thử độc lập các luật trò chơi.

Nội dung cài đặt được đối chiếu với `solvers/astar.py` và `solvers/common.py`.

2. Mô hình chi phí và cấu trúc dữ liệu

2.1. Quy tắc tính chi phí dùng chung

Gọi s là trạng thái trước, a là hành động và t là trạng thái sau. Chi phí chuyển trạng thái gồm chi phí chuyển động cơ sở, chi phí bổ sung trên các ô đích và chi phí thay đổi trạng thái cầu:

$$c(s, a, t) = e(s, a, t) + \sum_{x \in C(t)} p(x) + b(s, t).$$

Trong đó $C(t)$ là tập ô mà trạng thái đích chiếm, e là chi phí chuyển động cơ sở, p là chi phí bổ sung trên mỗi ô và b là chi phí thay đổi trạng thái cầu. Đây là mô hình định lượng của chương trình, không phải phép đo năng lượng vật lý. Các hằng số được giữ theo quy tắc tính chi phí được chuyển từ UCS sang `common.py`.

Bảng 2: Chi phí cơ sở của mỗi hành động.

Loại chuyển động	Chi phí cơ sở
Đứng → nằm hoặc kích hoạt tách khối từ tư thế đứng	1.0
Nằm → đứng, khi kết quả chưa tách	1.5
Nằm → nằm	1.2
Di chuyển từ trạng thái đã tách	0.8
`SPACE`: đổi khối lập phương đang điều khiển	0.3

Chi phí bổ sung gồm 0.5 cho mỗi ô dễ vỡ bị chiếm, 0.3 cho mỗi ô cầu bị chiếm, 0.1 cho mỗi ô đích kề khoảng trống theo phép kiểm tra bản đồ, và 0.5 nếu mặt nạ trạng thái cầu thay đổi. Phép kiểm tra kề khoảng trống xét bốn hướng; các ô cầu vẫn được xem là một phần của cấu trúc bản đồ. Chi phí trên các ô đích vẫn được tính cho `SPACE`, đúng theo cài đặt hiện tại. Thuật toán chỉ tính chi phí cho các trạng thái kế tiếp đã được phần lõi chấp nhận, không biến một nước đi thua thành nước đi hợp lệ.

Mọi chi phí bổ sung đều không âm, nên với quy tắc mặc định:

$$c(s, a, t) \geq c_{\min} = 0.3 > 0.$$

`StepCost(level)` tính trước chi phí bổ sung của các ô một lần cho mỗi lần giải. A* có thể nhận hàm tính chi phí tùy chỉnh; nếu không truyền kèm hàm ước lượng riêng, thuật toán mặc định dùng $h = 0$. Cận dưới 0.3 chỉ áp dụng cho quy tắc mặc định, không tự động áp dụng cho quy tắc khác.

2.2. Cấu trúc tìm kiếm

- **Hàng đợi ưu tiên:** dùng cấu trúc đống cực tiểu (min-heap), lưu các bộ `(f, counter, g, state)`.
- **Bảng chi phí tốt nhất:** `best_g[state]` lưu chi phí thấp nhất đã biết để tới một trạng thái.
- **Bảng cha:** `parents[state] = (previous_state, action)` dùng để dựng lời giải.
- **Bộ đếm thứ tự:** phân giải trường hợp hòa điểm theo thứ tự đưa vào hàng đợi, tránh phải so sánh trực tiếp các đối tượng trạng thái.

Không dùng riêng tọa độ làm khóa: hai trạng thái cùng vị trí nhưng khác trạng thái cầu hoặc khối đang điều khiển có thể có các bước đi tiếp theo khác nhau. Thuật toán không

khóa vĩnh viễn trạng thái đã mở rộng, mà cho phép xét lại khi tìm được đường đến có chi phí thấp hơn.

3. Thuật toán A* và cách tích hợp

3.1. Hàm ưu tiên

$$f(s) = g(s) + h(s).$$

$g(s)$ là chi phí thực tế của đường từ trạng thái bắt đầu tới s; $h(s)$ là cận dưới của chi phí còn lại. Hàng đợi ưu tiên lấy phần tử có f nhỏ nhất. Nếu hòa f, phần tử được đưa vào trước được xét trước, theo bộ đếm thứ tự và thứ tự sinh trạng thái kế tiếp của phần lõi.

3.2. Mã giả

Thuật toán 1: A* trên đồ thị trạng thái với chi phí không đều

Đầu vào: Màn chơi cố định, trạng thái bắt đầu, hàm chi phí và hàm ước lượng không vượt chi phí còn lại.
Đầu ra: Lời giải có chi phí tối ưu, hoặc thông báo không có lời giải khi hàng đợi cạn.

1: Chuẩn bị hàm chi phí; đặt `best_g[start] = 0`, cha của trạng thái bắt đầu là rỗng.
2: Đưa `(h(start), counter, 0, start)` vào hàng đợi.
3: Khi hàng đợi còn phần tử: lấy phần tử có ưu tiên nhỏ nhất.
4: Bỏ qua phần tử nếu `g` không còn bằng `best_g[state]`.
5: Tăng bộ đếm; nếu `is_goal(state, level)`, dựng lời giải và trả chi phí `g`.
6: Xét từng cặp `(action, child)` từ `get_successors(state, level)`.
7: Tính `new_g = g + cost(state, action, child, level)`.
8: Nếu `new_g` nhỏ hơn chi phí tốt nhất của `child`, cập nhật chi phí và cha.
9: Đưa `(new_g + h(child), counter, new_g, child)` vào hàng đợi, kể cả khi trạng thái đó đã mở rộng.
10: Nếu hàng đợi cạn, trả kết quả không có lời giải.

3.3. Loại phần tử cũ, mở lại trạng thái và điều kiện tối ưu

`heapq` không hỗ trợ giảm trực tiếp khóa ưu tiên của một phần tử đang có. Vì vậy khi cải thiện g, thuật toán đưa thêm phần tử mới vào hàng đợi và giữ phần tử cũ. Khi phần tử cũ được lấy ra, phép kiểm tra chi phí loại bỏ nó mà không tăng `expanded_nodes`.

Thuật toán chấp nhận đích khi lấy một phần tử còn hiệu lực khỏi hàng đợi, không phải ngay khi vừa sinh ra trạng thái đích. Việc mở lại trạng thái cho phép sử dụng hàm ước lượng chấp nhận được nhưng chưa nhất quán. Với chi phí dương hiện tại, hàm ước lượng không vượt chi phí còn lại và đồ thị trạng thái hữu hạn, thuật toán tìm được lời giải có chi phí tối ưu nếu lời giải tồn tại.

A* kiểm tra chi phí và giá trị ước lượng hữu hạn, chi phí dương, giá trị ước lượng không âm và bằng 0 tại đích. Các phép kiểm tra khi chạy không chứng minh được tính chấp nhận được của hàm ước lượng tùy chỉnh; người thiết kế hàm phải chứng minh điều kiện này.

3.4. Cách gọi từ chương trình

```
from core.interfaces import GameState
from levels.level_loader import load_level
from solvers.astar import AStarSolver
from solvers.common import run_solver

level = load_level("level_01")
start = GameState(level.start, level.initial_bridge_mask)
result = run_solver(AStarSolver(), level, start)
```

`solve()` tạo `SolverResult`; `run_solver()` bổ sung số đo thời gian và bộ nhớ. Trong giao diện, bộ kết nối chọn thuật toán và chạy hàm đo đạc trong tiến trình riêng. Luồng giao diện nhận kết quả để phát lại lời giải. Thao tác hủy hiện được thực hiện bằng cách dừng tiến trình tìm kiếm; thuật toán chưa có giới hạn thời gian, số nút hoặc cơ chế nhận yêu cầu hủy nội bộ. Các giao diện dự kiến trong tài liệu thiết kế không được xem là tính năng đã triển khai.

4. Thiết kế và chứng minh hàm ước lượng

4.1. Vì sao không dùng Manhattan cộng chi phí tư thế

Một lần lăn có thể làm tọa độ neo của khối dịch hơn một ô. Khi tách, khối còn có thể dịch chuyển tức thời tới vị trí xa. Vì vậy khoảng cách Manhattan từ tọa độ neo cộng một điểm cho tư thế nằm không mặc nhiên là cận dưới. Bản này dùng mô hình nới lỏng xét cả các đường dịch chuyển tức thời ngay khi trạng thái hiện tại chưa tách, như trình bày trong `docs/astar-heuristic.md`.

4.2. Mô hình điểm và các điểm mốc

Thay khối bằng một điểm trên lưới không giới hạn. Mỗi bước trong mô hình nới lỏng có chi phí 1 và đi tối đa 2 ô Manhattan, kể cả bước chéo (1,1). Mô hình bỏ qua tư thế, khoảng trống, ô dễ vỡ, trạng thái cầu và điều kiện kích hoạt công tắc. Mỗi công tắc tách khối có cạnh dịch chuyển tức thời có hướng tới từng vị trí đích của nó, với chi phí bằng 0.

Khoảng cách di chuyển thông thường giữa hai điểm trong mô hình là:

$$w(p, q) = \left\lceil \frac{|p_1 - q_1| + |p_2 - q_2|}{2} \right\rceil.$$

Chỉ giữ các điểm mốc gồm ô đích, vị trí công tắc tách khối và các vị trí nhận khối sau khi tách. Nối mọi cặp bằng w, thêm cạnh dịch chuyển tức thời với chi phí 0, rồi chạy Dijkstra trên đồ thị đảo từ ô đích. Gọi $d(p)$ là khoảng cách từ điểm p tới ô đích trong mô hình nới lỏng. Với điểm bất kỳ, khoảng cách bằng giá trị nhỏ nhất của chi phí đi tới một điểm mốc cộng khoảng cách còn lại của điểm mốc đó. Cách rút gọn này vẫn xét mọi chuỗi dịch chuyển tức thời: một đường đi hoặc tới thẳng ô đích, hoặc phải tới đầu của cạnh dịch chuyển tức thời đầu tiên.

Với $C(s)$ là tập một hoặc hai ô mà trạng thái thật đang chiếm, đặt:

$$H(s) = \min_{p \in C(s)} d(p), \quad h(s) = 0.3H(s).$$

Khi đã tách, phép lấy giá trị nhỏ nhất xét cả hai khối lập phương, không chỉ khối đang điều khiển. Một trạng thái chưa thắng vẫn có thể có $h = 0$, chẳng hạn khối nằm đè lên ô đích; đây vẫn là cận dưới hợp lệ.

4.3. Chứng minh tính nhất quán và tính chấp nhận được

Tính nhất quán yêu cầu $h(s) \le c(s, a, t) + h(t)$ trên mỗi cạnh hợp lệ. Tính chấp nhận được yêu cầu hàm ước lượng không vượt quá chi phí tối ưu còn lại. Xét cạnh thật $s \to t$ có chi phí $c(s, a, t)$. Chọn ô q thuộc $C(t)$ sao cho $d(q) = H(t)$. Có một ô p thuộc $C(s)$ với đường đi tới q trong mô hình nới lỏng có chi phí tối đa 1:

- **Lăn khối:** theo bảng chuyển tư thế, mỗi ô mới cách ít nhất một ô cũ không quá 2 đơn vị Manhattan.
- **Di chuyển khối lập phương:** khối đang điều khiển dịch 1 ô, khối còn lại đứng yên.
- **Đổi điều khiển bằng SPACE:** tập ô không đổi nên có thể mô phỏng bằng đường đi có chi phí 0.
- **Ghép khối:** tập ô sau ghép trùng với tập ô của hai khối lập phương sau chuyển động.
- **Tách và dịch chuyển tức thời:** từ ô cũ tới công tắc trong tối đa một bước nới lỏng, rồi tới vị trí nhận khối với chi phí 0.
- **Cầu và công tắc:** thay đổi mặt nạ trạng thái không ảnh hưởng đến lập luận vì mô hình bỏ các ràng buộc cản đường.

Suy ra $H(s) \le d(p) \le 1 + d(q) = 1 + H(t)$. Vì chi phí thật ít nhất 0.3:

$$h(s) \le 0.3 + h(t) \le c(s, a, t) + h(t).$$

Vậy hàm ước lượng nhất quán theo quy tắc chi phí hiện tại. Mọi chi phí trong mô hình nới lỏng không âm nên $h(s) \ge 0$; tại trạng thái đích, tập ô chỉ chứa ô đích nên giá trị ước lượng bằng 0. Cộng bất đẳng thức nhất quán dọc một đường tới đích cho thấy giá trị ước lượng không lớn hơn chi phí của đường đó. Do đó hàm cũng chấp nhận được.

4.4. Chi phí tính toán và giới hạn

Với K điểm mốc, đồ thị đầy đủ có $O(K^2)$ cạnh. Cài đặt Dijkstra với hàng đợi ưu tiên dùng khoảng $O(K^2 \log K)$ thời gian tiền xử lý. Bộ nhớ đệm lưu $O(K)$ khoảng cách cho mỗi cấu hình; hàng đợi tạm có thể chứa $O(K^2)$ phần tử. Mỗi lần đánh giá xét tối đa hai ô đang chiếm và K điểm mốc, cộng chi phí tạo khóa từ các công tắc tách khối. Bộ nhớ đệm LRU giữ tối đa 32 cấu hình ô đích và các cạnh dịch chuyển tức thời.

Giới hạn của hàm ước lượng

Hàm ước lượng bỏ nhiều ràng buộc nên có thể yếu khi lời giải cần phối hợp hai khối lập phương hoặc mở cầu. Không bảo đảm luôn giảm số nút hay thời gian so với $h = 0$. Khi quy tắc chi phí thay đổi, phải cập nhật cận dưới và chứng minh lại; không tự động dùng hệ số 0.3 cho mọi quy tắc tùy chỉnh.

5. Thực nghiệm và đánh giá

5.1. Mục tiêu và cách bố trí thực nghiệm

Chương trình `typst/collect_results.py` so sánh A* mặc định với cùng bản cài đặt dùng $h = 0$. Phương án $h = 0$ ưu tiên trạng thái có chi phí tích lũy nhỏ nhất; đây là phương

án đối chứng có thứ tự tìm kiếm như UCS, không phải bản cài đặt UCS độc lập. Cả hai dùng cùng `StepCost`, cùng màn chơi và trạng thái bắt đầu. Mục tiêu là kiểm tra tính hợp lệ của lời giải, đối chiếu chi phí và đo ảnh hưởng của hàm ước lượng tới tài nguyên tìm kiếm.

Các màn `level_01` đến `level_10` được đọc trực tiếp từ thư mục `levels/`. Mỗi cấu hình được chạy thử một lần không ghi nhận số đo, rồi đo 5 lần. Hai cấu hình chạy tuần tự và đổi thứ tự qua các lần lặp để giảm ảnh hưởng của thứ tự chạy. Mỗi lần dùng một đối tượng thuật toán mới. Bảng báo cáo lấy trung vị riêng của từng chỉ số theo màn và cấu hình, không lấy số đo của một lần chạy duy nhất.

Môi trường thu thập: Python 3.12.1 trên `Windows-11-10.0.26200-SP0`. Tổng số lần đo là 100. Tệp `data/environment.json` lưu cách đo, phiên bản môi trường và mã băm SHA-256 của chương trình thu thập, mã nguồn thuật toán, phần lõi và các màn. Tệp `data/raw_runs.csv` giữ số liệu từng lần; `data/validation.csv` giữ số liệu tổng hợp dùng trong các bảng dưới. Cần thu thập lại sau khi thay mã nguồn.

5.2. Cách đo thời gian và bộ nhớ

Cả hai cấu hình đều gọi `run_solver()` trong `solvers/common.py`, cùng hàm đo mà giao diện sử dụng. Thời gian được đo bằng `perf_counter()` quanh `solve()`, theo đơn vị mili giây. Khoảng đo bao gồm tiền xử lý chi phí, xây dựng dữ liệu hàm ước lượng phát sinh trong lần giải và dựng lời giải; không gồm đọc màn, thu gom rác trước khi chạy, vẽ giao diện hay phát lại.

Trước mỗi lần đo, chương trình xóa bộ nhớ đệm của hàm ước lượng. Do đó số liệu A* bao gồm chi phí khởi tạo bộ nhớ đệm trong lần giải đó. Bộ nhớ đệm chỉ số cầu của phần lõi được chuẩn bị trước cho cả hai cấu hình. Chương trình gọi thu gom rác trước mỗi lần đo; cơ chế thu gom tự động của Python vẫn hoạt động khi thuật toán chạy.

Bộ nhớ được ghi nhận bằng `tracemalloc.get_traced_memory()[1]`, là mức cấp phát Python cao nhất trong lần giải. `run_solver()` chia số byte cho 1024 và lấy phần nguyên; bảng giữ nhãn KB theo trường dữ liệu hiện tại. Đây không phải toàn bộ bộ nhớ của tiến trình hay mức dùng RAM của trò chơi. Mỗi lần có một phiên đo riêng, không chạy chồng các phép đo trong cùng tiến trình. Việc theo dõi cấp phát có thể làm tăng thời gian chạy, và ảnh hưởng đó áp dụng cho cả hai cấu hình.

5.3. Kiểm chứng lời giải và số nút mở rộng

Ngoài khoảng đo, chương trình phát lại từng lời giải bằng `apply_move`, kiểm tra từng hành động hợp lệ, trạng thái cuối thỏa `is_goal` và tổng chi phí tính lại khớp kết quả. Tính ổn định của kết quả qua các lần lặp cũng được kiểm tra. Nếu có sai lệch hoặc lỗi thực thi, chương trình dừng và không công bố bộ CSV mới chưa hoàn tất.

Bảng 3: Đối chiếu chi phí và số nút mở rộng.

Màn	Bước A*	Chi phí A*	Chi phí h=0	Nút A*	Nút h=0
level_01	10	13.3	13.3	65	68
level_02	9	13.1	13.1	36	41
level_03	9	13.5	13.5	21	24
level_04	7	10.5	10.5	32	37
level_05	12	19.3	19.3	62	65
level_06	12	18.3	18.3	40	42
level_07	12	17.6	17.6	30	36
level_08	9	12.2	12.2	1193	1444
level_09	11	15.3	15.3	2965	3354
level_10	18	24.5	24.5	3924	3925

Số nút là số lần lấy phần tử còn hiệu lực khỏi hàng đợi, gồm đích và các lần mở lại trạng thái; không đếm phần tử cũ bị bỏ qua. Hai cấu hình cho cùng chi phí trên các màn đã xét, và lời giải đều được phát lại hợp lệ. Cột số bước chỉ ghi lời giải A*: số bước của phương án đối chứng có thể khác khi tồn tại nhiều đường có cùng chi phí.

5.4. Kết quả thời gian và bộ nhớ

Bảng 4: Trung vị thời gian và bộ nhớ đỉnh qua 5 lần đo mỗi cấu hình.

Màn	A* (ms)	h=0 (ms)	A* (KB)	h=0 (KB)
level_01	10.239	10.349	29	29
level_02	11.974	11.516	22	22
level_03	5.365	6.731	13	14
level_04	5.093	4.883	17	18
level_05	9.779	8.462	27	27
level_06	8.395	7.637	21	20
level_07	6.682	7.264	17	18
level_08	351.515	366.986	739	790
level_09	1019.392	1027.887	1646	1739
level_10	1308.798	1170.841	1845	1812

Trong đợt đo này, A* mở rộng ít nút hơn trên 10/10 màn, có trung vị thời gian thấp hơn trên 5/10 màn và có trung vị bộ nhớ thấp hơn trên 5/10 màn. Các nhận xét này được tính trực tiếp từ bảng kết quả của lần thu thập hiện tại.

Giảm số nút không bảo đảm giảm thời gian tương ứng: A* còn phải tính hàm ước lượng, tạo khóa cấu hình và tra bộ nhớ đệm. Trên màn nhỏ, chi phí này có thể lớn hơn phần tìm kiếm tiết kiệm được. Hàm ước lượng bỏ nhiều ràng buộc và dùng hệ số bảo thủ 0.3, nên ở một số màn mức giảm tìm kiếm còn nhỏ. Số nút giảm cũng không quyết định duy nhất mức dùng bộ nhớ vì còn có dữ liệu điểm mốc, bảng chi phí, bảng cha và các phần tử cũ trong hàng đợi.

Vì sao số đo có thể khác khi chơi trên giao diện?

Giao diện thường hiển thị kết quả của một lần chạy từ trạng thái người chơi đang đứng. Báo cáo lấy trung vị nhiều lần từ trạng thái ban đầu, xóa bộ nhớ đệm hàm ước lượng trước mỗi lần và chạy ngoài giao diện. Dù cùng dùng `run_solver()`, điều kiện chạy khác nhau có thể cho số đo khác.

5.5. Giới hạn và hướng hoàn thiện

Phạm vi kết luận

Kết quả áp dụng cho các màn, trạng thái bắt đầu, mã nguồn và môi trường được ghi nhận. Số liệu không chứng minh A* luôn nhanh hơn hoặc dùng ít bộ nhớ hơn. Phương án đối chứng dùng chung mã tìm kiếm, nên chưa loại trừ lỗi chung của bản cài đặt hoặc thay thế đối chiếu với UCS độc lập.

Chương trình thu thập hiện chưa đặt giới hạn thời gian hoặc số nút, và chưa khảo sát nhiều máy, nhiều trạng thái giữa màn hay các màn không có lời giải được thiết kế riêng. Trước khi kết luận cho toàn bộ đồ án, nhóm cần bổ sung các thuật toán độc lập, phân nhóm màn theo cơ chế và giữ cùng quy tắc đo. BFS tối ưu số hành động, còn UCS và A* tối ưu chi phí chung; không so sánh chất lượng lời giải chỉ bằng số bước.

Trước khi nộp, cần điền thông tin bìa của báo cáo chung, xác nhận đóng góp của các thành viên và ghi nhận hỗ trợ AI theo quy định môn học. Bản thảo được soạn với trợ lý AI từ mã nguồn, tài liệu hàm ước lượng và dữ liệu đo; người thực hiện cần rà soát và xác nhận nội dung.

Tài liệu tham khảo